

Conformally Certified Sketching and Shape-Aware Sparse Analytics for Reproducible GraphChallenge Anonymized Network Sensing

Qifan Chen

University of California, Santa Barbara
qifan_chen@ucsb.edu

Abstract—The MIT/IEEE/Amazon GraphChallenge Anonymized Network Sensing track asks participants to construct and analyze anonymized source-to-destination traffic matrices from packet-derived data. We present a commodity-CPU toolkit built around two ideas. First, an empirical finding with a sparse-recovery explanation: candidate-tracking capacity, not Count-Min sketch width, bounds heavy-hitter discovery on diffuse traffic – width buys measurements, capacity buys decoding – and a SpaceSaving retention bound computed from a 50,000-record prefix predicts the capacity scale that sweeps confirm; a second decoder family (dyadic Count-Min descent) reproduces the same identifiability gradient. Second, *conformal identifiability certificates*: distribution-free lower bounds on future-window ranking gaps that decide whether a memory budget certifies exact top- k identity, with adaptive conformal inference maintaining target coverage under drift. The conformal certificate correctly refuses a 655 KB budget that the single-prefix screen had accepted, and prices distribution-free certification at 10.5 MB on a synthetic hotspot regime and 21 MB on real CTU-13 traffic. We further validate against the official challenge data products: byte-range prefixes of the official Random PCAP (40,000,000 packets parsed in seconds), whose all-tie structure the diagnostics correctly report as unidentifiable, and the official GraphBLAS matrices, which our parser reproduces exactly (64 of 64 windows); a SuiteSparse:GraphBLAS baseline places the transparent SciPy kernel within 25% of the optimized library. The contribution is not full-capture throughput, but a reproducible, student-oriented bridge between official GraphChallenge materials and certified commodity sparse-matrix experimentation.

I. INTRODUCTION

The Anonymized Network Sensing Graph Challenge frames packet-derived records as sparse source-to-destination traffic matrices for graph and linear-algebra analysis [1]; submissions are paper-based and reward measured innovation across software, hardware, algorithms, and systems [2], [3]. This submission targets a complementary point in that design space: not maximum throughput, but a reproducible, low-dependency baseline that makes traffic-shape dependence and approximation risk measurable – and certifiable.

To be explicit about what kind of paper this is: it is a diagnostics-and-certification contribution, not a performance record. The central empirical finding is that candidate-tracking capacity, not Count-Min sketch width, bounds heavy-hitter discovery on diffuse traffic; a sparse-recovery reading (width buys measurements, capacity buys decoding) explains it and predicts that it is decoder-independent, which we verify. The

contributions are layered around that finding. The *certification layer* – the primary contribution – turns traffic-shape measurements into guarantees: a sufficient-condition sketch-safety screen with a SpaceSaving retention bound that quantitatively predicts the required capacity, a single identifiability index I_k , and *conformal identifiability certificates* that replace the prefix-representativeness assumption with distribution-free coverage over stream windows, including adaptive conformal inference under drift. The *validation layer* supports the diagnostics with multi-regime synthetic benchmarks over equivalence-checked exact baselines, a real capture (CTU-13), byte-range prefixes of the official Random PCAP product, an exact cross-check against the official GraphBLAS matrices, and a SuiteSparse:GraphBLAS baseline. The *toolkit layer* – capability, not headline – adds an early-stream method selector, time-bin anomaly summaries, official-format checks, and scripts that regenerate every table and figure in this paper.

II. RELATED WORK AND POSITIONING

GraphChallenge traffic-matrix tasks are closely aligned with sparse linear algebra: GraphBLAS defines graph analytics as sparse matrix operations [4], SuiteSparse:GraphBLAS implements them efficiently [5], and recent reference-implementation work improves the official pipeline [6]. Our work is complementary: a student-facing CPU toolkit with regime controls, failure diagnostics, and certification, not a replacement for high-performance reference code.

Network measurement has also long used sketches and heavy-hitter algorithms for high-rate streams, including Count-Min Sketch [7], Misra-Gries summaries [8], SpaceSaving/top- k stream tracking [9], and sketching systems for network measurements [10]. Sketches are linear measurements of a sparse vector, so heavy-hitter discovery is a sparse-recovery problem [11]; we use this correspondence to interpret our results. On the uncertainty side, conformal prediction provides distribution-free finite-sample guarantees [12], has been extended to drifting data [13], and has been applied to frequency estimation from sketched data [14]; we conformalize a different object – the top- k identifiability decision – rather than per-query frequency intervals. Traffic anomaly studies show that feature distributions and traffic shape can be as important as aggregate volume [15]. This package connects those ideas to

the GraphChallenge setting while keeping the claim boundary conservative.

A. Student Innovation

The student innovation is not a hardware record but a certification-first reproducibility package for commodity CPUs, intentionally not optimized with GPU, C++, or distributed GraphBLAS – a baseline with guarantees rather than a performance ceiling.

III. APPROACH

The input format is an anonymized edge table with source identifier, destination identifier, and a numeric traffic weight such as bytes or packets. The pipeline factorizes source and destination labels, constructs a SciPy coordinate matrix, converts it to compressed sparse row format, and sums duplicate source-destination pairs. This yields a sparse matrix $A \in \mathbb{R}^{m \times n}$ where A_{ij} is the aggregate traffic from source i to destination j .

The implementation then computes network-sensing analytics: matrix size, nonzeros, density, total traffic, top- k heavy-hitter pairs, source and destination degree and strength distributions, entropies of source strength, destination strength, and edge weights, a largest singular value as a lightweight spectral summary, and a Gini coefficient of nonzero edge weights as a concentration statistic.

The repository includes three exact construction methods. The primary method builds a sparse matrix directly from edge records. A pandas baseline first groups by source and destination. A pure Python dictionary baseline aggregates pairs with a counter-like loop for small inputs. All exact methods are checked for output equivalence on the same synthetic inputs.

The new streaming path processes edge records sequentially. It keeps exact source and destination strength summaries, a fixed-size Count-Min Sketch for approximate edge weights [7], and a bounded candidate tracker inspired by streaming heavy-hitter algorithms [8], with lazy-deletion heap eviction so that even all-unique streams cost $O(\log c)$ per record. The streaming path is intentionally evaluated as an online approximation rather than a faster replacement for optimized SciPy sparse construction. Its main diagnostic is top- k heavy-hitter recall against the exact sparse matrix, which isolates candidate discovery from weight estimation and exposes the capacity bound reported in Section VII.

As a toolkit capability, the time-bin path groups the same edge records by a supplied `time_bin` field, computes per-bin volume, nonzeros, entropy, Gini, and top-share statistics, and scores bins with a robust positive-deviation summary; temporal aggregation can expose burst, scan, or concentration patterns that disappear in a single summed matrix.

Finally, the online selector uses only an early stream prefix to compute duplication rate, estimated nonzero growth, source and destination entropy, edge Gini, top-edge share, and top source/destination concentration. It applies the rule table below and is meant to make traffic-shape dependence visible, not to replace profiling.

Early-stream selector. From a prefix it computes duplication rate, nnz growth, edge Gini, and top shares, recommending pandas pre-aggregation when the prefix is concentrated and sparse direct construction otherwise, with fixed-candidate sketching marked unsafe for diffuse prefixes. Its thresholds are empirical and chosen only to separate the synthetic regimes here; Section VII reports a real-traffic miss, which is why the certification layer, not this heuristic, is the primary contribution.

IV. CONSERVATIVE SCREENING CONDITIONS

The safety screen adds conservative sufficient conditions to the empirical selector rule. We state its components and assumptions explicitly: it is a prefix-computed screen, not a distribution-free guarantee about the full stream.

For exact construction, let N be edge records, U unique source-destination pairs, and $\rho = U/N$. A first-order model $T_{\text{sparse}} = \alpha_s N + \beta_s U$, $T_{\text{pandas}} = \alpha_p N + \beta_p U$ predicts that pandas pre-aggregation wins only below a shape threshold in ρ ; the threshold is hardware-dependent, so this is intuition, not a law.

For sketch safety, the candidate tracker is SpaceSaving with weighted updates [9]: an arriving tracked key adds its weight; an untracked key replaces a minimum-count key and inherits that count plus its weight. By the standard SpaceSaving/Misra-Gries analysis [8], [9], every tracked count overestimates its key’s true weight by at most $W/(c+1)$ for stream weight W and capacity c , and every key with true weight above $W/(c+1)$ is tracked. Let w_k be the k th largest aggregated prefix edge weight and $\Delta_k = w_k - w_{k+1}$ the ranking gap; we define top- k identity only when $\Delta_k > 0$, and the screen refuses when $\Delta_k = 0$ (ties), as on the official prefix below.

The screen assumes: (A1) *prefix representativeness* – weight shares and rank order measured on the prefix carry over to the stream (checked empirically across eight disjoint official-capture segments in Section VII, but not guaranteed); (A2) Count-Min with width m , depth d overestimates any queried weight by at most ϵW , $\epsilon = e/m$, with probability at least $1 - e^{-d}$ per query [7]. Under (A1)–(A2), three sufficient conditions follow. *Retention*: $c \geq W/w_k - 1$ ensures every true top- k pair is tracked. *Ranking*: $\Delta_k > 2\epsilon W$ ensures Count-Min estimates preserve the top- k boundary among tracked keys. *Combined*: the implemented screen requires the stricter $\Delta_k > 2(\epsilon W + W/(c+1))$, which additionally covers rankings read from tracker counts rather than sketch estimates. Failure of the screen does not prove sketch failure; satisfaction certifies top- k retention and separation under the stated assumptions only.

A. A Sparse-Recovery View and Identifiability

Count-Min is a linear sketch with a sparse binary measurement matrix, and heavy-hitter discovery is a sparse-recovery problem [11]: sketch width buys measurements (it controls estimation noise ϵW), while candidate capacity or tree-search budget is the *decoder*. This view predicts the central finding of Section VII rather than merely describing it: on concentrated

traffic the aggregated weight vector is effectively sparse and any reasonable decoder recovers it, while all-tie traffic is a maximally flat signal that no decoder can identify from any number of measurements, because top- k membership carries no information. We summarize a stream-budget pair by a single dimensionless *top- k identifiability index*

$$I_k = \frac{\Delta_k}{2(\epsilon W + W/(c+1))}, \quad (1)$$

so that $I_k \geq 1$ is exactly the combined screen condition. Because $w_k \geq \Delta_k$ always, $I_k \geq 1$ implies the retention condition as well, so one number decides the certificate. Section VII tests the decoder-independence prediction directly by comparing SpaceSaving tracking against dyadic Count-Min tree descent [7], a group-testing-style decoder that needs no per-record candidates.

B. Conformal Identifiability Certificates

The plug-in screen above trusts one prefix (A1). We remove that assumption with conformal prediction [12]: split the stream into disjoint equal windows, treat them as exchangeable, and score each by its gap share $S_i = \Delta_k^{(i)}/W^{(i)}$. With m calibration windows and $r = \lfloor \alpha(m+1) \rfloor \geq 1$, let L be the r th smallest calibration score; the order-statistic argument gives $\Pr[S_{\text{future}} < L] \leq \alpha$. *Proposition:* if $L > 2(\epsilon + 1/(c+1))$, then with probability at least $1 - \alpha$ a future window's top- k is retained and correctly separated by the budget (given A2 per query) – the combined condition applied to $S_{\text{future}} \geq L$. The certificate is distribution-free over windows and prices the removal of A1 in memory. Because drift breaks exchangeability, we also run adaptive conformal inference [13], which adjusts α_t online. Unlike conformal frequency estimation over sketches [14], which calibrates per-query frequency intervals, the conformalized object here is the *identifiability decision itself*.

V. REPRODUCIBILITY PACKAGE

The package is intentionally small: setup, sample-generation, and sparse-analysis scripts; benchmark and figure scripts for the exact, streaming, sensitivity, selector, screen, conformal, and dyadic experiments; a time-bin anomaly script; an official-data prefix fetcher that Range-downloads the first N compressed bytes of the official `pcap.zst`, stream-decompresses the truncated `zstd` frame, parses the fixed-size packet records, and emits an anonymized `src,dst,bytes` edge table with a provenance manifest; a real-trace adapter; and a local GraphChallenge-paper-style formula comparison.

The synthetic generator is not a substitute for CAIDA-scale data. It provides four regimes: `hotspot_zipf`, `community_bursty`, `scanner_fanout`, and `uniform_sparse`. It also exposes controlled knobs for edge count, source and destination counts, target nonzeros or density, duplication rate, Zipf skew, community structure, scanner fanout, and temporal bursts. These controls make the local benchmark runnable without protected downloads or multi-gigabyte packet captures while still testing multiple sparse traffic shapes.

VI. EXPERIMENTS

Experiments were run on a MacBook Air with Apple M3 CPU, 8 cores, 16 GB RAM, macOS 26.5.1 arm64, Python 3.9.6, NumPy 2.0.2 with Apple Accelerate BLAS/LAPACK, SciPy 1.13.1, pandas 2.3.3, matplotlib 3.9.4, and zstandard 0.25.0. No GPU, distributed runtime, or manual BLAS thread pinning is used.

The official-data experiments use the two official challenge data products published in the GraphChallenge S3 bucket: the Random PCAP (13.9 GB `pcap.zst`) and the Random GraphBLAS matrices (8.6 GB tar); these are the organizer-generated products in official formats, while the CAIDA-telescope-derived variant is access-controlled and not used here. An HTTP Range request fetches the first 256 MiB (later extended to 2 GiB) of the `pcap.zst`, the truncated `zstd` frame is stream-decompressed, and 5,000,000 (respectively 40,000,000) fixed-size packet records (RAW-IP link type, 40-byte IPv4/TCP headers) are parsed with a vectorized NumPy path in under five seconds per 5,000,000 packets. Identifiers are the challenge-provided anonymized addresses used as opaque integer labels; no re-anonymization is applied. A provenance manifest records the URL, byte range, parser path, packet counts, and output checksum. One 67 MB inner tar of the GraphBLAS product (the first 2^{23} -packet segment) is fetched the same way for the exact cross-check, deserialized with `python-graphblas` on SuiteSparse:GraphBLAS. The small benchmark uses 5,000–100,000-record edge tables with three repeats and three exact methods; the large benchmark uses 1,000,000 and 5,000,000 records, three repeats with the minimum reported, and two scalable methods. Runtimes exclude synthetic generation and disk I/O; the input table is memory-resident. The streaming benchmark uses 25,000–500,000 records; width and capacity sensitivity studies use 100,000. Selector and screen experiments use the first 50,000 records of each 5,000,000-edge stream; conformal experiments use 50,000-record windows (56–100 per stream) split half calibration, half test. The real capture is CTU-13 scenario 42. Time-bin experiments inject four controlled anomaly types, and an official-format smoke test round-trips a Matrix Market matrix. Commands are exposed through the repository Makefile.

The key correctness check is output equivalence. For every benchmark row, the matrix produced by the method under test is compared with the sparse direct matrix. Total traffic is also compared. A benchmark row is marked valid only when both checks pass.

VII. RESULTS

All numbers and figures in this section were generated on the machine described in Section VI, and the repository Makefile regenerates every table and figure from saved CSV outputs.

Table I reports the 5,000,000-edge large benchmark. The regimes differ substantially: hotspot traffic has 222,097 nonzeros and high concentration, while uniform sparse traffic has

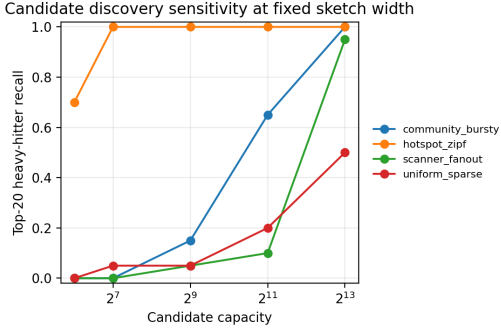


Fig. 1. Candidate-capacity sensitivity at fixed sketch width. Candidate capacity, rather than sketch width, is the limiting factor for diffuse heavy-hitter discovery.

TABLE I
LARGE BENCHMARK AT 5,000,000 INPUT EDGES (MINIMUM OF THREE REPEATS; STANDARD DEVIATION AT MOST 0.02 S PER CELL). RUNTIME EXCLUDES SYNTHETIC GENERATION AND DISK I/O.

Regime	nnz/edge	Sparse	Sp. M/s	pandas	pd. M/s
Hotspot	0.044	0.142	35.2	0.110	45.3
Community	0.451	0.285	17.6	0.590	8.5
Scanner	0.992	0.377	13.3	1.189	4.2
Uniform	0.999	0.366	13.7	1.205	4.1

4,997,173 nonzeros and density 0.0011. Sparse direct construction is faster than pandas groupby in community, scanner, and uniform regimes. In the hotspot regime, pandas groupby is faster on this machine, which is a useful negative result: the best construction path depends on traffic shape and duplication rate. Throughput scales nearly linearly from 1,000,000 to 5,000,000 edges (sparse direct moves from 14.1–32.7 to 13.3–35.2 million edges per second across regimes), so the reported rates are not a small-input artifact. The table reports total measured construction plus analytics time and throughput for in-memory edge tables.

The streaming/sketch result at 500,000 edges (327,680-byte Count-Min, capacity 512, 3.2–3.5 s per stream): top-20 recall is 1.00 on hotspot with median relative error 0.0003, but 0.10 on community and 0 on scanner and uniform. A width sweep from 512 to 32,768 reduces hotspot median weight error from 0.0405 to 0 yet does not recover a single missed diffuse heavy hitter at fixed candidate capacity. Figure 1 shows the missing factor: raising candidate capacity from 64 to 8,192 lifts community recall from 0 to 1.00 and scanner recall from 0 to 0.95. With the lazy-deletion heap tracker this costs almost nothing at run time (0.62 s to 0.71–0.91 s at 100,000 edges), so capacity is a memory knob, not a latency penalty. In this Python implementation streaming remains slower than SciPy sparse construction; its value is bounded-memory online processing, not raw speed.

The online method-selector experiment: from only the first 50,000 records the selector matches the saved 5,000,000-edge exact-method winner in all four synthetic regimes (pandas for the high-duplication hotspot, sparse direct for the three near-

unit-growth regimes; per-regime features are in the repository CSVs). It is an online decision rule with visible failure boundaries – but it misses on real traffic below, which is why certification, not the heuristic, is the primary contribution.

A. Official-Dataset Validation

The candidate-capacity finding and the online diagnostics were checked on the official challenge data products. Table II summarizes the experiment on the 5,000,000-packet prefix of the official Random PCAP. The prefix is an extreme diffuse regime: every packet is a unique source-destination pair (duplication rate 0), there are 4,997,115 unique anonymized sources, and every packet is a 40-byte header-only record, so all edge weights are equal – consistent with the randomized generation the organizers describe. On this input the early-stream selector, using only the first 50,000 records, recommends sparse direct construction and flags fixed-candidate sketching as unsafe; the measured end-to-end construction winner (including label factorization) is sparse direct (2.52 s versus 5.15 s for pandas groupby, about 2.0×10^6 edges per second), so the recommendation matches. The safety screen withholds certification because the prefix top- k ranking gap is exactly zero. The capacity sweep confirms the diagnosis from the other direction: strict-identity top-20 recall is exactly zero at capacities 64 through 8,192, while tie-aware recall (any equal-weight candidate matches) is trivially 1.0 – the divergence between the two definitions is itself the diagnosis: top- k identity carries no information in an all-tie regime. This is the complementary case to the synthetic diffuse regimes: there, capacity rescues recall because heavy hitters exist but are hard to discover; here, the screen correctly reports that sketch-based top- k is ill-posed. Extending the fetch to 2 GiB (40,000,000 packets) and repeating the analysis on eight disjoint 5,000,000-packet segments yields identical diagnoses on all eight, so this is not an artifact of the first window.

Two further checks tie the toolkit to the official GraphBLAS ecosystem. First, an exact cross-check: the official GraphBLAS product stores one $2^{32} \times 2^{32}$ UINT32 matrix per 2^{17} -packet window; deserializing the first 2^{23} -packet segment (64 matrices) and comparing coordinate sets against our pcap-parsed edge table yields an exact match in 64 of 64 windows (8,388,608 pairs, after the bijective host/network byte-order relabeling) – our parser reproduces the official matrix pipeline without running the official code. Second, a SuiteSparse:GraphBLAS baseline on the same 5,000,000 edges, with construction time decomposed to avoid ambiguity: label factorization 2.1 s (shared by both compact paths; this dominates the end-to-end times in Table II); SciPy COO→CSR aggregation kernel 0.106 s; GraphBLAS `from_coo` kernel 0.086–0.088 s; and GraphBLAS `from_coo` directly in the native $2^{32} \times 2^{32}$ hypersparse address space, 0.086 s end to end with no factorization. The transparent SciPy kernel is within about 25% of the optimized library, and the native hypersparse mode is an architectural advantage that SciPy CSR cannot replicate, since its row-pointer array alone would need tens of gigabytes.

TABLE II
OFFICIAL RANDOM PCAP PREFIX EXPERIMENT (5,000,000 PACKETS
PARSED FROM A 256 MiB BYTE-RANGE REQUEST IN UNDER FIVE
SECONDS).

Quantity	Value
Packets parsed	5,000,000
Unique src-dst pairs (dup. rate)	5,000,000 (0.0)
Sparse direct, end-to-end (s)	2.52
pandas groupby, end-to-end (s)	5.15
Measured winner	sparse_direct
Selector recommendation	sparse_direct (match)
Safety-screen decision	not certified (gap = 0)
Recall@20 strict / tie-aware	0.00 / 1.00

Table IV reports compatibility validation rather than a speed contest. The Matrix Market smoke test reloads a GraphBLAS-style sparse coordinate matrix and exactly matches the edge-table sparse matrix on shape, nonzeros, and total weight. The formula check compares our pipeline with a local implementation of GraphChallenge-paper-style sparse sensing formulas on a 100,000-edge community sample. This is not the full official GraphChallenge reference repository, but it verifies that core scalar quantities agree before performance claims are interpreted.

B. Real Traffic, Conformal Certificates, and Decoders

We add a real capture – CTU-13 scenario 42 (2,824,636 bidirectional flows, 697,518 unique pairs, duplication 0.75, addresses relabeled to opaque integers) – and report three experiments that close the loop on Assumption A1 and on the sparse-recovery view.

First, honesty about the heuristic selector: on CTU-13 its concentration features (edge Gini 0.98) trigger the pandas recommendation, but sparse direct measures faster (0.087 s versus 0.175 s) – the selector’s first miss, illustrating why hand-tuned thresholds are fragile. The sketch-safety side is unaffected: top-20 recall is 0.95 at capacity 64 and 1.00 from 512 up, at median relative error 1.6×10^{-4} .

Second, conformal certificates (Table III). Calibrating on the first half of each stream’s 50,000-record windows and testing on the rest shows why one prefix should not be trusted: the hotspot window-level gap varies by two orders of magnitude, fixed calibration under-covers on drifting streams (empirical coverage as low as 0.82 at target 0.90), and adaptive conformal inference restores realized miscoverage to 0.10–0.11 on every source. The conformal certificate consequently *refuses* the 655 KB budget the plug-in screen had certified for hotspot – the first window was optimistic – and prices distribution-free certification honestly: hotspot at width 2^{18} , capacity 9.8×10^4 (10.5 MB); CTU-13 at width 2^{19} , capacity 2×10^5 (≈ 21 MB); the diffuse regimes at none of the tested budgets; the all-tie official product never (gap identically zero). Removing A1 is not free; the certificate reports its memory price.

Third, decoder independence. Replacing per-record Space-Saving tracking with dyadic Count-Min tree descent – a group-testing-style decoder with no candidate capacity – reproduces

TABLE III
CONFORMAL IDENTIFIABILITY CERTIFICATES AT $\alpha = 0.1$ OVER
50,000-RECORD WINDOWS. $L_{0.1}$ IS THE CONFORMAL LOWER BOUND ON
THE FUTURE-WINDOW GAP SHARE; TUNED IS THE 655 KB BUDGET THE
PLUG-IN SCREEN CERTIFIED FOR HOTSPOT; PRICE IS THE SMALLEST
TESTED BUDGET THAT CERTIFIES DISTRIBUTION-FREE.

Source	$L_{0.1}$	Tuned	Price
Hotspot	4.4×10^{-5}	no	10.5 MB
CTU-13 (real)	3.1×10^{-5}	no	21 MB
Community	1.1×10^{-7}	no	>21 MB
Scanner	1.4×10^{-7}	no	>21 MB
Uniform	8.4×10^{-8}	no	>21 MB
Official (all-tie)	0	no	impossible

TABLE IV
COMPATIBILITY VALIDATION FOR OFFICIAL-FORMAT AND
REFERENCE-FORMULA PATHS.

Check	Input	nnz	Weight	Status
Matrix reload	10k MM	4,747	66.1M	match
Formula scalars	100k edge	75,952	648.3M	match
Official .grb	2 ²³ pkt	8.39M	8.39M	64/64 exact

the same identifiability gradient: recall (tracker/dyadic) is 1.00/0.95 on hotspot, 1.00/0.95 on CTU-13, degraded on the diffuse regimes (0.15/0.40, 0.05/0.25, 0.05/0.15), and exactly 0.00/0.00 on the all-tie official product. Discovery is bounded by signal structure, not by the choice of data structure, as the sparse-recovery view predicts.

Finally, the time-bin capability: in all four controlled scenarios (volume burst, scanner fanout, concentrated destination, distributed low-rate scan), the injected bin is the top-ranked bin by the robust anomaly score, and a different feature explains each event type. We report this as a toolkit capability rather than a core contribution; per-scenario curves and numbers are in the repository figures and CSVs.

The selector separates exact-method choice from heuristic sketch use: pandas pre-aggregation is recommended only when early traffic is highly duplicated and concentrated, while fixed-candidate sketching is marked unsafe for all three diffuse regimes even when exact sparse construction remains fast.

Table V reports the safety screen of Section IV from the same 50,000-record prefix under two sketch budgets. Under the default width 8,192 and candidate capacity 512, no regime satisfies the sufficient conditions. The screen is not a permanent refusal: raising the budget to width 16,384 and candidate capacity 5,000 (a 655 KB sketch) satisfies the combined condition for the hotspot regime with margin $+5.6 \times 10^{-5}$, while the diffuse regimes remain uncertified. Notably, the rigorous retention bound now predicts the capacity scale that the empirical sweep independently confirms: the screen computes $c_{\min} \approx 2,000$ for the community regime, where measured recall reaches 0.65 at capacity 2,048 and 1.00 at 8,192, and $c_{\min} \approx 5,800$ for the scanner regime, where recall reaches 0.95 only at 8,192. The screen therefore tells an operator both when sketch safety is not proven and what budget would prove it. Applied to the official-capture prefix, it withholds certification

TABLE V

SAFETY SCREEN FROM A 50,000-RECORD PREFIX UNDER TWO BUDGETS: DEFAULT (WIDTH 8,192, CAPACITY 512) AND TUNED (WIDTH 16,384, CAPACITY 5,000). TOP-20 IS THE CUMULATIVE SHARE OF THE TWENTY HEAVIEST PAIRS; w_{20}/W IS THE SHARE OF THE 20TH PAIR ALONE; $c_{\min} = \lceil W/w_{20} \rceil - 1$ IS THE SPACE SAVING RETENTION BOUND; $\text{WIDTH}_{\min}$ IS THE COUNT-MIN WIDTH REQUIRED BY THE GAP CONDITION.

Regime	Top-20	w_{20}/W	$c_{\min}$	$\text{Width}_{\min}$	Def.	Tuned
Hotspot	.352	.0058	173	6.9k	no	yes
Community	.011	.00050	1,990	2.4M	no	no
Scanner	.004	.00017	5,839	2.5M	no	no
Uniform	.004	.00015	6,654	24.3M	no	no

because the ranking gap is zero, matching the all-tie shape reported above.

The current benchmark supports three conservative observations. First, all local exact construction paths produce equivalent aggregate matrices on the tested inputs, which is necessary before performance comparisons are meaningful. Second, the sparse path provides a concise implementation that naturally exposes interpretable sensing summaries without densifying the traffic matrix. Third, multi-regime, online, and time-bin results matter: a method or sketch that looks best on high-duplication hotspot traffic may not dominate when nnz approaches the edge count, early stream statistics can steer exact-method choice, and temporal bursts can be easier to explain through bin-level summaries than through a single aggregate matrix.

VIII. LIMITATIONS

This submission deliberately avoids overclaiming. The official-data experiments use byte-range prefixes (up to 40,000,000 packets), not a full 2^{30} -packet run; prefix composition may differ from the full product (the eight-segment check mitigates, not eliminates, this); and the Random products are organizer-generated synthetic traffic – the CAIDA-telescope variant is access-controlled and unused, so no claim is made about real telescope traffic. The conformal certificates provide marginal, not conditional, coverage; validity rests on window exchangeability, which drift violates – fixed calibration then under-covers, adaptive conformal inference restores target mis-coverage but trades the finite-sample guarantee for asymptotic tracking; and with 50–100 windows per stream, levels below $\alpha \approx 0.02$ are unresolvable. The real-traffic evidence is one CTU-13 scenario. Memory measurements are process-level; runtimes exclude disk I/O and synthetic generation (fetch and parse times are in the committed manifest). A same-machine comparison against the official reference workflow remains future work, and the synthetic generator is a reproducibility tool, not a validated traffic model.

ARTIFACT AVAILABILITY

The complete repository – source, tests, Makefile, result CSVs, and provenance manifests with output checksums behind every table and figure – is available at <https://github.com/QifanChen62/graphsense-network-sensing>. `make`

`paper-numbers` regenerates every table and figure; `make fetch-official-prefix` re-fetches and re-parses the official prefix against the committed SHA-256.

IX. CONCLUSION

We presented a reproducible toolkit for anonymized network sensing whose core is certification: a sufficient-condition safety screen with a SpaceSaving retention bound, a top- k identifiability index, and conformal identifiability certificates that remove the prefix-representativeness assumption at a measured memory price – 10.5 MB for a synthetic hotspot regime, 21 MB for real CTU-13 traffic, and provably never for the all-tie official product. The central empirical result, explained by the sparse-recovery view and reproduced by two decoder families, is that candidate capacity rather than sketch width bounds heavy-hitter discovery on diffuse traffic, at the scale the retention bound predicts. The toolkit also shows that traffic regime changes the right construction method, that heuristic selector thresholds miss on real traffic, and that fixed conformal calibration under-covers on drifting streams unless adapted online; the parser reproduces the official GraphBLAS matrices exactly. These failure cases are part of the contribution: they make approximation risk visible and priced. The package is a bridge between official GraphChallenge materials and certified commodity-CPU experimentation.

REFERENCES

- [1] “Anonymized network sensing graph challenge,” arXiv:2409.08115, 2024.
- [2] “MIT/IEEE/Amazon GraphChallenge,” <https://graphchallenge.mit.edu/>, accessed 2026-06-29.
- [3] “GraphChallenge Submission Instructions,” <https://graphchallenge.mit.edu/submit>, accessed 2026-06-29.
- [4] A. Buluç, T. Mattson, S. McMillan, J. Moreira, and C. Yang, “The graphblas c api specification,” in *IEEE International Parallel and Distributed Processing Symposium Workshops*, 2017.
- [5] T. A. Davis, “Algorithm 1000: Suitesparse:graphblas: Graph algorithms in the language of sparse linear algebra,” *ACM Transactions on Mathematical Software*, 2019.
- [6] I. Voloshchuk, H. Jananthan, C. Byun, and J. Kepner, “Improving the graph challenge reference implementation,” arXiv:2601.00974, 2026.
- [7] G. Cormode and S. Muthukrishnan, “An improved data stream summary: The count-min sketch and its applications,” *Journal of Algorithms*, 2005.
- [8] J. Misra and D. Gries, “Finding repeated elements,” *Science of Computer Programming*, 1982.
- [9] A. Metwally, D. Agrawal, and A. El Abbadi, “Efficient computation of frequent and top-k elements in data streams,” in *International Conference on Database Theory*, 2005.
- [10] A. Kumar, J. Xu, J. Wang, O. Spatscheck, and L. Li, “Sketching data streams for network measurements,” in *Proceedings of the ACM SIGCOMM Conference*, 2004.
- [11] A. Gilbert and P. Indyk, “Sparse recovery using sparse matrices,” *Proceedings of the IEEE*, vol. 98, no. 6, 2010.
- [12] V. Vovk, A. Gammernan, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.
- [13] I. Gibbs and E. Candès, “Adaptive conformal inference under distribution shift,” in *Advances in Neural Information Processing Systems*, 2021.
- [14] M. Sesia and S. Favaro, “Conformal frequency estimation with sketched data,” in *Advances in Neural Information Processing Systems*, 2022.
- [15] A. Lakhina, M. Crovella, and C. Diot, “Mining anomalies using traffic feature distributions,” in *Proceedings of the ACM SIGCOMM Conference*, 2005.